Mediapipe Gesture-AprilTag ID Sorting

1. Function Description

After the program runs, the robotic arm will lift up, then recognize gestures. The gestures are 1-4 (corresponding to the number of extended fingers), and then the robotic arm will move to the recognition posture. It will recognize AprilTags on the desktop. If the robotic arm recognizes an AprilTag with an ID corresponding to the gesture, it will lower its gripper to grasp it. If not, it will lift the robotic arm for the next gesture recognition.

2. Function Startup

After entering the docker container, enter the following commands in the docker terminal:

```
# Start USB camera node and inverse kinematics node
ros2 launch dofbot_info camera_arm_kin.launch.py
# Start gesture recognition node
ros2 run dofbot_sorting mediapipe_gesture
# Start AprilTag ID recognition node
ros2 run dofbot_sorting apriltag_gesture_id
# Start grasping node
ros2 run dofbot_sorting grasp
```

After successfully running the program, the robotic arm will move to its initial position, then lift up and enter the gesture recognition posture. At this time, we can give gestures 1-4 by extending the corresponding number of fingers. After the program accumulates 40 recognitions and the gesture has not changed, it will publish the gesture recognition result and the robotic arm will move to the AprilTag recognition posture to start recognizing AprilTags. As shown in the figure below, place the AprilTag wooden blocks within the robotic arm's camera view.

Then press the spacebar to start recognition:

- If there is an AprilTag corresponding to the gesture ID (i.e., there exists an AprilTag wooden block with the same ID as the gesture), the robotic arm will lower its gripper to grasp it, then place it at the designated position, and finally return to the AprilTag recognition posture for the next recognition;
- If there is no AprilTag wooden block with the same ID as the gesture or no AprilTag wooden blocks are recognized, the robotic arm will move to the gesture recognition posture for the next gesture recognition.

3. Code Analysis

3.1. mediapipe_gesture

After entering the docker container, the source code path is located at:

```
/root/dofbot_sorting/dofbot_sorting/mediapipe_gesture.py
```

Import necessary libraries:

```
import cv2
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import cv2 as cv
from std_msgs.msg import Bool,Int16
import time
# Defined mediapipe library, source code location is at
/root/dofbot_ws/src/dofbot_sorting/dofbot_sorting/media_library.py
from dofbot_sorting.media_library import *
from rclpy.node import Node
import rclpy
import threading
from Arm_Lib import Arm_Device
```

Create robotic arm control object and control the robotic arm to move to gesture recognition posture:

```
self.Arm = Arm_Device()
self.detect_joints = [90, 150, 12, 20, 90, 0]
self.Arm.Arm_serial_servo_write6_array(self.detect_joints,2000)
```

Create hand detection object:

```
self.hand_detector = HandDetector()
```

Create publishers and subscribers:

```
# Create publisher to publish gesture recognition result topic
self.pub_GesturetId = self.create_publisher(Int16,"GesturetId",1)
# Create subscriber to subscribe to reset gesture topic node
self.subscription =
self.create_subscription(Bool,'/reset_gesture',self.get_resetCallBack,1)
# Create subscriber to subscribe to image topic messages
self.subscription =
self.create_subscription(Image,'/image_raw',self.ImageCallback,1)
```

Reset gesture topic node callback function get_resetCallBack:

```
def get_resetCallBack(self,msg):
    if msg.data == True:
        # Set self.pub_gesture value to True, indicating that the next gesture
recognition result can be published
        self.pub_gesture = True
        self.last_sum = 0
        self.cnt = 0
        # Control robotic arm to return to gesture recognition posture
        self.Arm.Arm_serial_servo_write6_array(self.detect_joints,2000)
```

Image topic callback function ImageCallback:

```
def ImageCallback(self,color_msg):
    # Process received image topic messages, convert to image
    rgb_image = self.rgb_bridge.imgmsg_to_cv2(color_msg, "bgr8")
    # Call process function, passing in the image
    self.process(rgb_image)
```

process function:

```
def process(self, frame):
    # Call findHands function in hand detection object to get hand information
    frame, lmList, bbox = self.hand_detector.findHands(frame)
    # If hand is detected and self.pub_gesture is True, it means gesture
recognition result information can be published
    if len(lmList) != 0 and self.pub_gesture == True:
        # Create thread and execute function Gesture_Detect_threading, passing
hand information as parameters
        gesture = threading.Thread(target=self.Gesture_Detect_threading, args=
(lmList,bbox))
        gesture.start()
        gesture.join()
    self.cTime = time.time()
    fps = 1 / (self.cTime - self.pTime)
    self.pTime = self.cTime
    text = "FPS : " + str(int(fps))
    cv.putText(frame, text, (20, 30), cv.FONT_HERSHEY_SIMPLEX, 0.9, (0, 0, 255),
1)
    if cv.waitKey(1) & 0xFF == ord('q'):
        cv.destroyAllWindows()
    cv.imshow('frame', frame)
```

Gesture_Detect_threading gesture recognition function:

```
def Gesture_Detect_threading(self, lmList,bbox):
    # Call fingersUp function in hand detection object to get the number of
extended fingers
    fingers = self.hand_detector.fingersUp(lmList)
    print("sum of fingers: ",sum(fingers))

    print(self.pub_gesture).
    # If the current number of extended fingers is consistent with the last
recorded gesture, it means the gesture has not changed
    if sum(fingers) == self.last_sum:
        print("----------------------------")
        # Increment recognition count
        self.cnt = self.cnt + 1
        print("cnt: ",self.cnt)
        # When accumulated recognition count equals 10, gesture is considered
unchanged and self.pub_gesture is True, indicating gesture recognition result
topic can be published
        if self.cnt==10 and self.pub_gesture == True:
            print("sum of fingers: ",self.last_sum)
            # Set self.pub_gesture result to False to prevent multiple repeated
publications
            self.pub_gesture = False
            # Create gesture recognition result topic message object
            sum_gesture = Int16()
```

```
            # Assign data to gesture recognition result topic message object,
value is the last recorded gesture
            sum_gesture.data = self.last_sum
            self.pub_GesturetId.publish(sum_gesture)
            self.Arm.Arm_serial_servo_write6_array(self.grasp_joints,2000)
    else:
        self.cnt = 0
    # Update last recognized gesture, value is the number of extended fingers
    self.last_sum = sum(fingers)
```

3.2. apriltag_gesture_id

After entering the docker container, the source code path is located at:

```
/root/dofbot_sorting/dofbot_sorting/apriltag_gesture_id.py
```

Import necessary libraries:

```
import rclpy
from rclpy.node import Node
import cv2
import numpy as np
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import cv2 as cv
from dt_apriltags import Detector
from dofbot_sorting.vutils import draw_tags
from dofbot_interface.srv import Kinemarics
from dofbot_interface.msg import *
from std_msgs.msg import Bool,Int16
import time
from Arm_Lib import Arm_Device
import os
from dofbot_sorting.compute_joint5 import *
```

Create robotic arm control object and control the robotic arm to move to initial position:

```
self.Arm = Arm_Device()
self.init_joints = [90.0, 120.0, 0.0, 0.0, 90.0, 0.0]
self.Arm.Arm_serial_servo_write6_array(self.init_joints,2000)
```

Create publishers and subscribers:

```
# Create publisher to publish recognized position information topic messages
self.pos_info_pub = self.create_publisher(AprilTagInfo, "PosInfo",
qos_profile=10)
# Create subscriber to subscribe to grasp completion topic messages
self.sub_grasp_done =
self.create_subscription(Bool,'grasp_done',self.GraspStatusCallback,qos_profile=
1)3
# Create subscriber to subscribe to gesture recognition result topic
self.sub_gestureID =
self.create_subscription(Int16,"GesturetId",self.GestureIDCallback,1)
# Create publisher to publish reset gesture topic messages
self.pub_reset = self.create_publisher(Bool, "/reset_gesture", qos_profile=10)
# Create publisher to publish calculated joint 5 angle reference value topic
messages
self.pub_joint5 = self.create_publisher(Int16, "set_joint5", 10)
```

Grasp completion topic callback function GraspStatusCallback:

```
def GraspStatusCallback(self,msg):
    print("**")
    if msg.data == True:
    # Assign self.pubPos_flag value to True, indicating that position information
topic can be published
        self.pubPos_flag = True
        self.detect_flag = False
```

Gesture recognition result topic callback function:

```
def GestureIDCallback(self,msg):
    self.target_id = msg.data
    time.sleep(5.0)
```

Image topic callback function ImageCallback:

```
def ImageCallback(self,color_frame):
    # Process received image topic messages, convert to image
    rgb_image = self.rgb_bridge.imgmsg_to_cv2(color_frame,'rgb8')
    result_image = np.copy(rgb_image)
    tags = self.at_detector.detect(cv2.cvtColor(rgb_image, cv2.COLOR_RGB2GRAY),
False, None, 0.025)
    tags = sorted(tags, key=lambda tag: tag.tag_id) # Seems to be already in
ascending order, no need for manual sorting
    draw_tags(result_image, tags, corners_color=(0, 0, 255), center_color=(0,
255, 0))
    self.Center_x_list = list(range(len(tags)))
    self.Center_y_list = list(range(len(tags)))
    key = cv2.waitKey(10)
    if key == 32:
        self.pubPos_flag = True
    # Find AprilTag
    if len(tags) > 0 :
        # Traverse AprilTag list
        for i in range(len(tags)):
            # If current AprilTag ID value equals target AprilTag ID value
            if tags[i].tag_id == self.target_id:
```

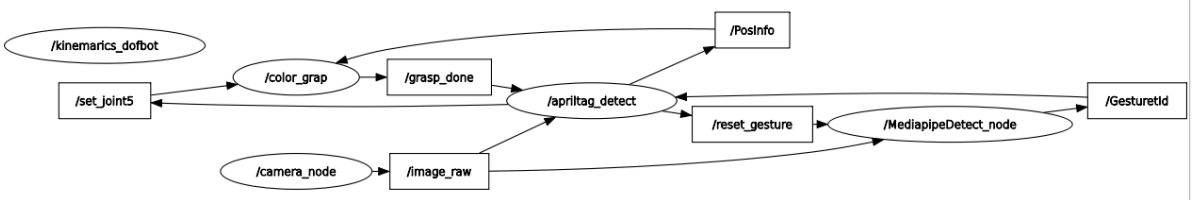
```
                # Set self.detect_flag value to True, indicating target AprilTag
was found
                self.detect_flag = True
                self.pubPos_flag = True
                # Extract current AprilTag center point coordinates
                center_x, center_y = tags[i].center
                cx = center_x
                cy = center_y
                cz = self.dist
                # Based on minimum enclosing circle center coordinates, convert
coordinates to normalized coefficients a and b. a is x-axis offset coefficient, b
is y-axis offset coefficient. The calculated ba values are the center point
coordinates in the world coordinate system, which will eventually be sent to the
sorting grasp node.
                (a, b) = (round(((320 - cx) / 4000), 5), round(((480 - cy) /
3000) * 0.8+0.15, 5))
                print("a: ",a)
                print("b: ",b)
                # Calculate joint 5 angle reference value based on corner offset
angle
                vx = int(tags[i].corners[0][0]) - int(tags[i].corners[1][0])
                vy = int(tags[i].corners[0][1]) - int(tags[i].corners[1][1])
                target_joint5 = compute_joint5(vx,vy)
                print("target_joint5: ",target_joint5)
                if self.pubPos_flag == True:
                    # Assign self.pubPos_flag value to False to prevent multiple
repeated publications of position information topic
                    self.pubPos_flag = False
                    joint5 = Int16()
                    joint5.data = int(target_joint5)
                    self.pub_joint5.publish(joint5)
                    # Create position information topic message object
                    pos = AprilTagInfo()
                    pos.x = b
                    pos.y = a
                    pos.z = 0.03
                    pos.id = tags[i].tag_id
                    # Publish position information topic
                    self.pos_info_pub.publish(pos)
        # If target ID AprilTag was not found and current target AprilTag ID is
not 0, publish reset gesture topic message
        if self.detect_flag == False and self.target_id !=0:
            reset = Bool()
            reset.data = True
            self.pub_reset.publish(reset)
            self.pubPos_flag = False

    else:
        # If spacebar was pressed and no AprilTag was found, publish reset
gesture topic message
        if self.pubPos_flag == True:
            print("Do not find any tags")
            reset = Bool()
            reset.data = True
            self.pub_reset.publish(reset)
            self.pubPos_flag = False
```

3.3. grasp

Already analyzed in [13.3D Sorting and Grasping Course]-[1. AprilTag ID Sorting]-[3. Code Analysis], you can refer to the content there.

4. Node Communication Diagram

After entering the docker container, enter the following commands in the docker terminal:

```
ros2 run rqt_graph rqt_graph
```